

大语言模型及应用

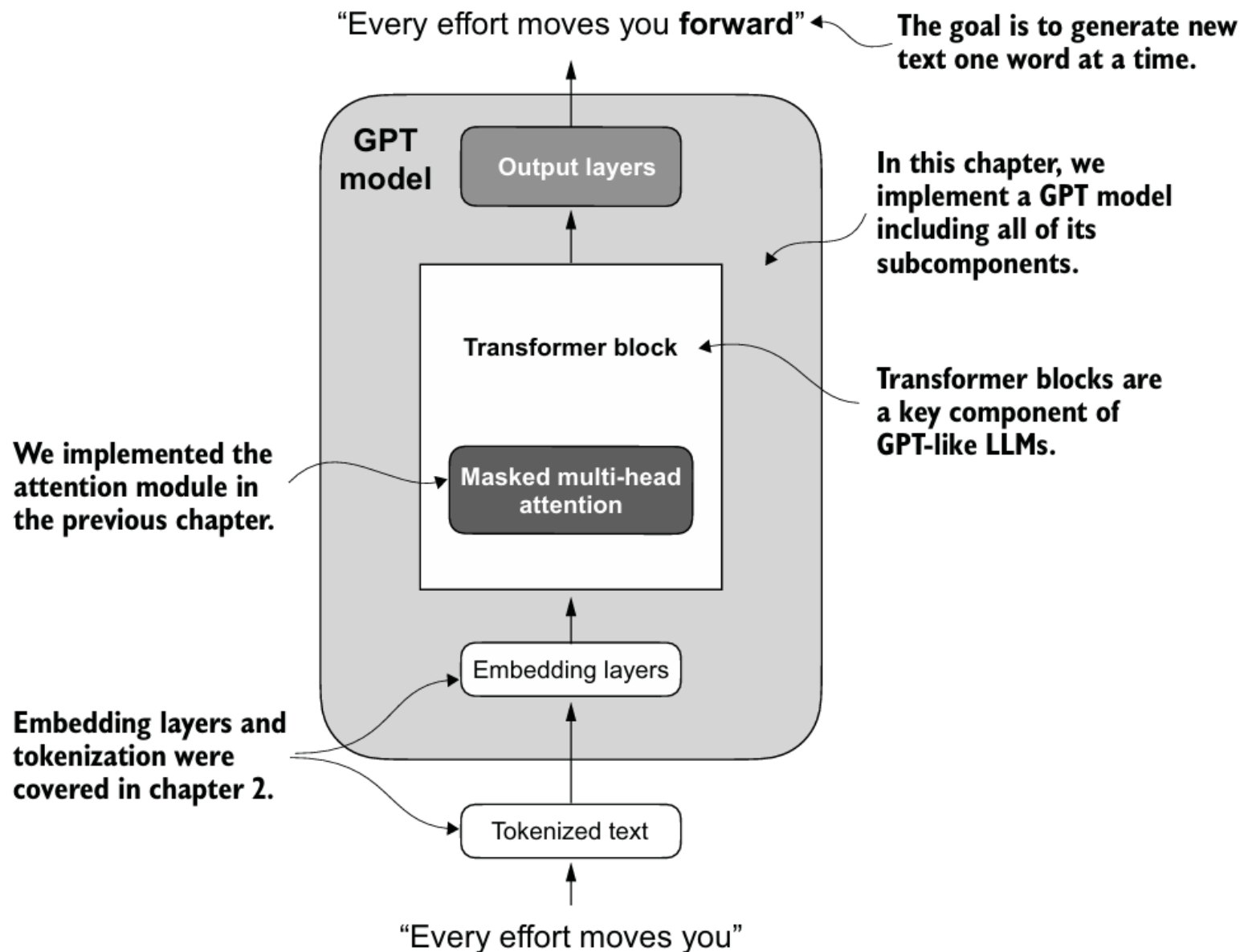
Instructor: Xiaohan Ma(马晓寒), Ph.D.

Graduate School of Information and Communication Technology,

Ajou University, Korea

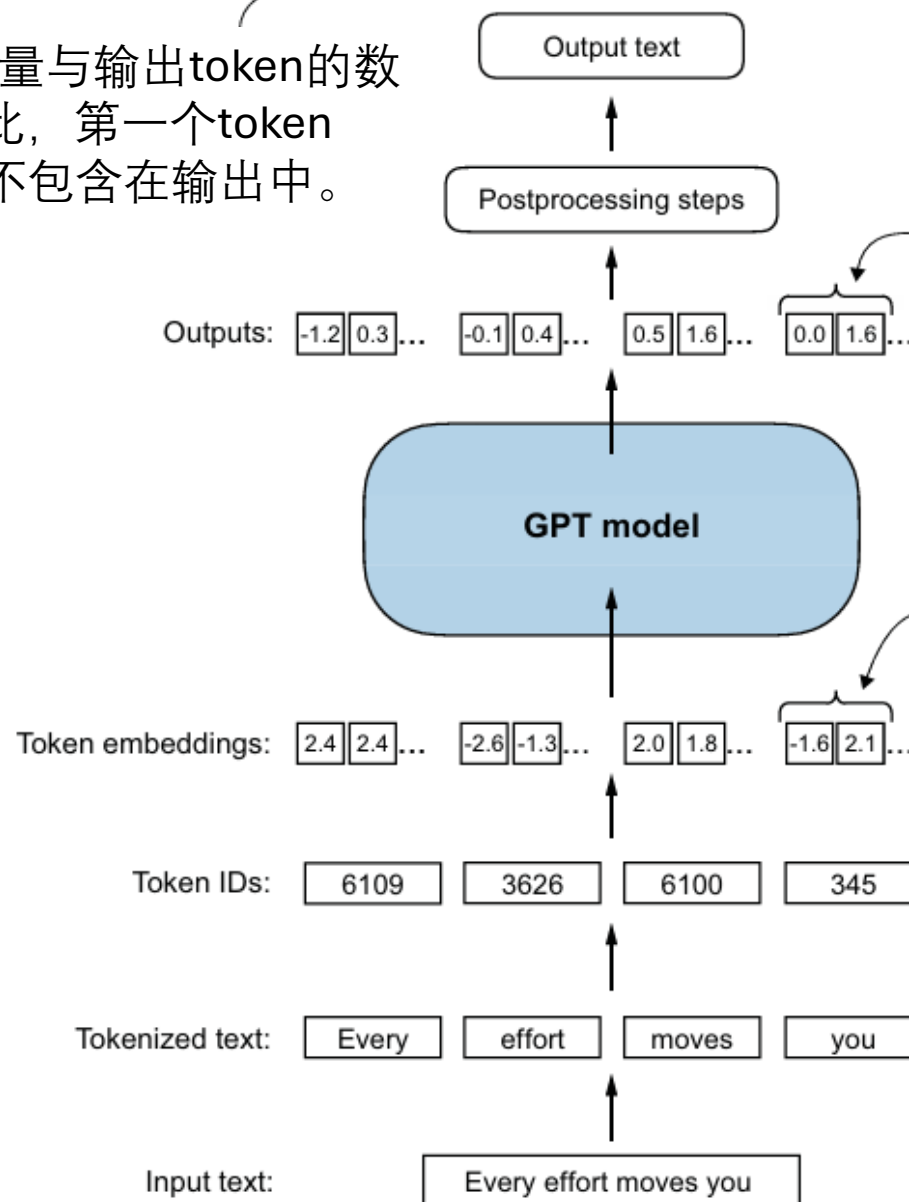
maxiaohan@ajou.ac.kr

大语言模型架构



目标是生成下一个单词“forward”。

输入token的数量与输出token的数量相匹配；因此，第一个token（“Every”）将不包含在输出中。

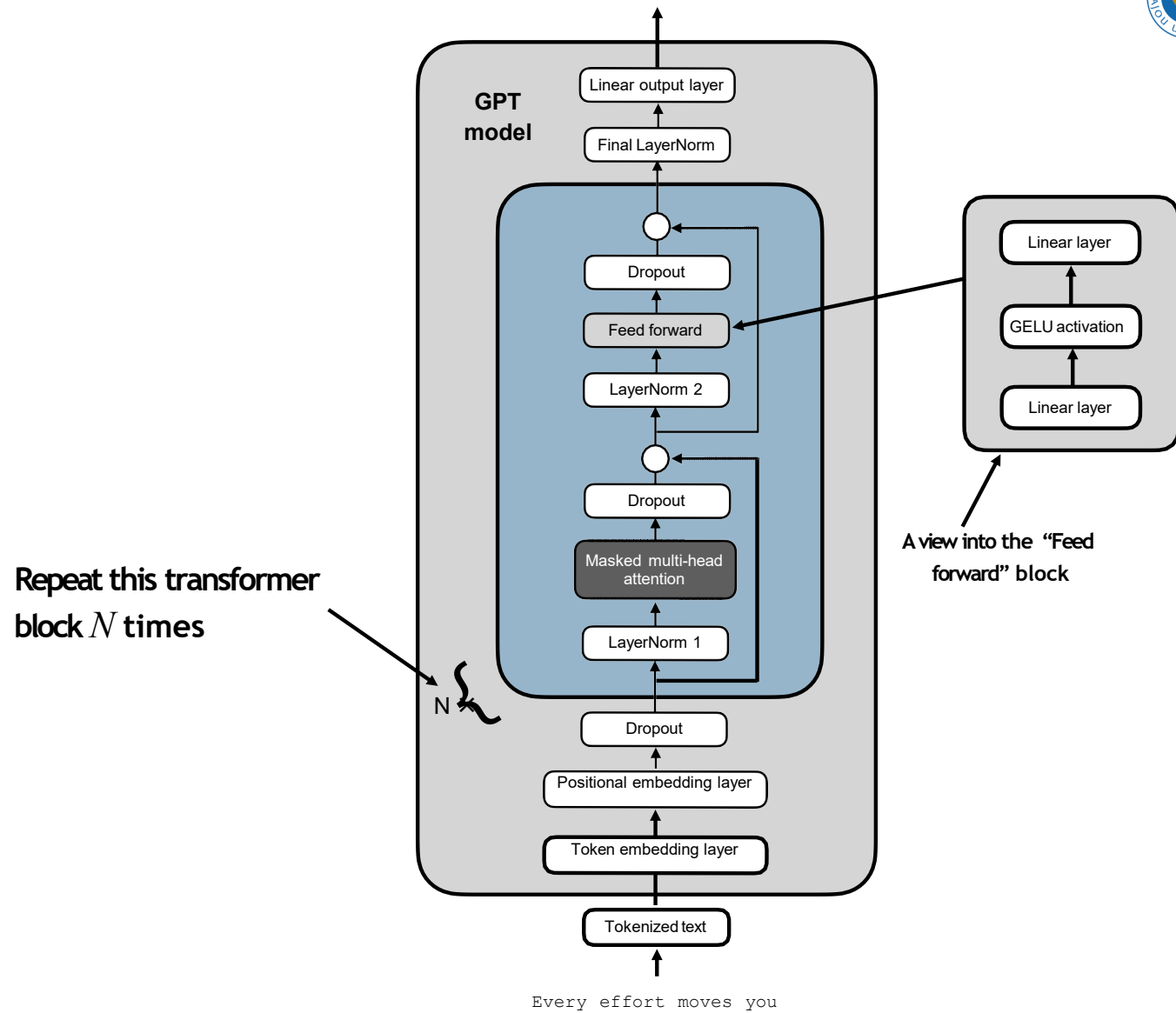


LLM 为每个输入token(词向量维度为768) 返回一个 768 维输出向量。

对于最小的 GPT-2 模型，每个嵌入向量由 768 个维组成（仅显示前 2 个维）。

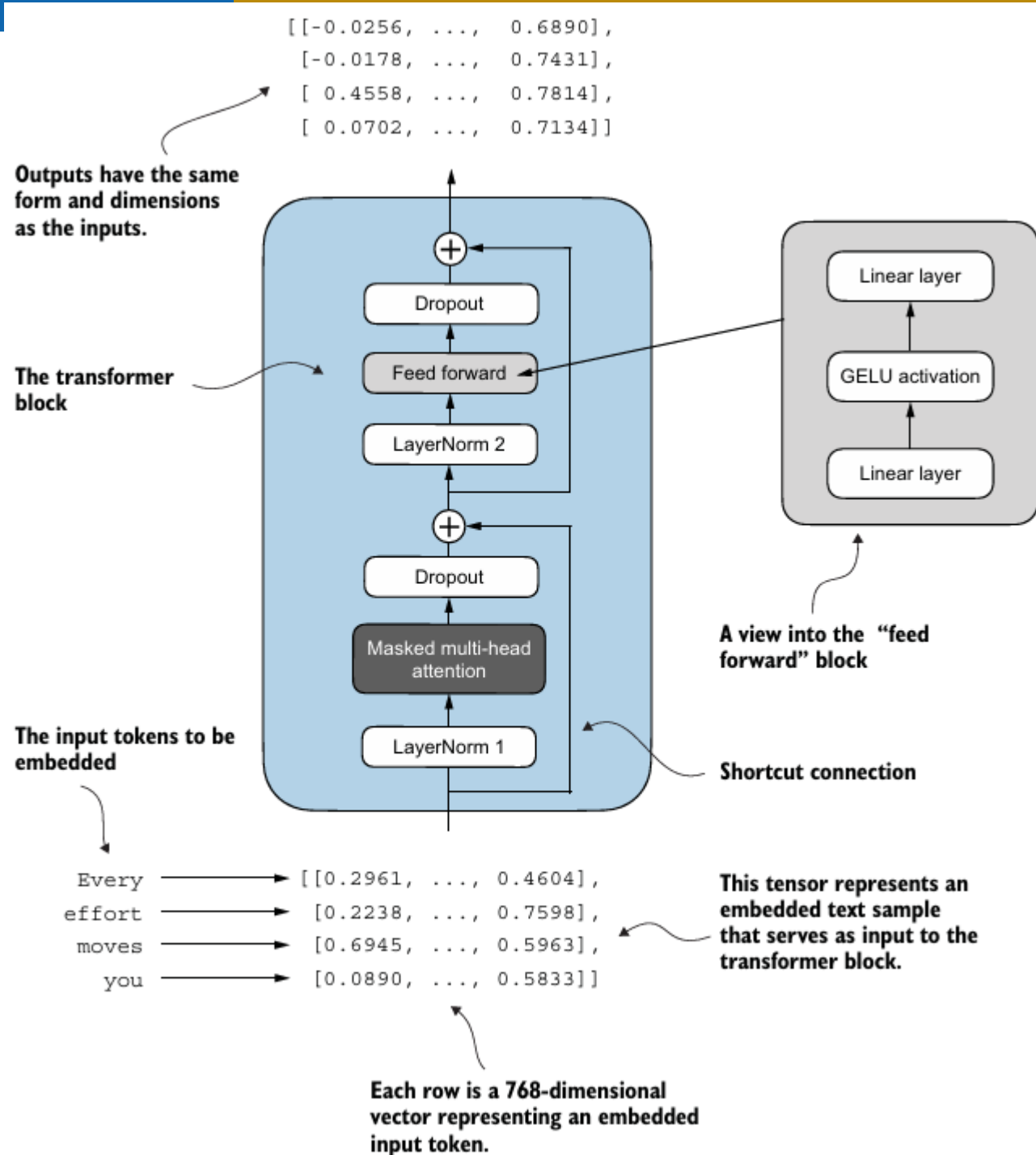
我们对输入文本进行标记并将其转换为标记嵌入。

GPT model



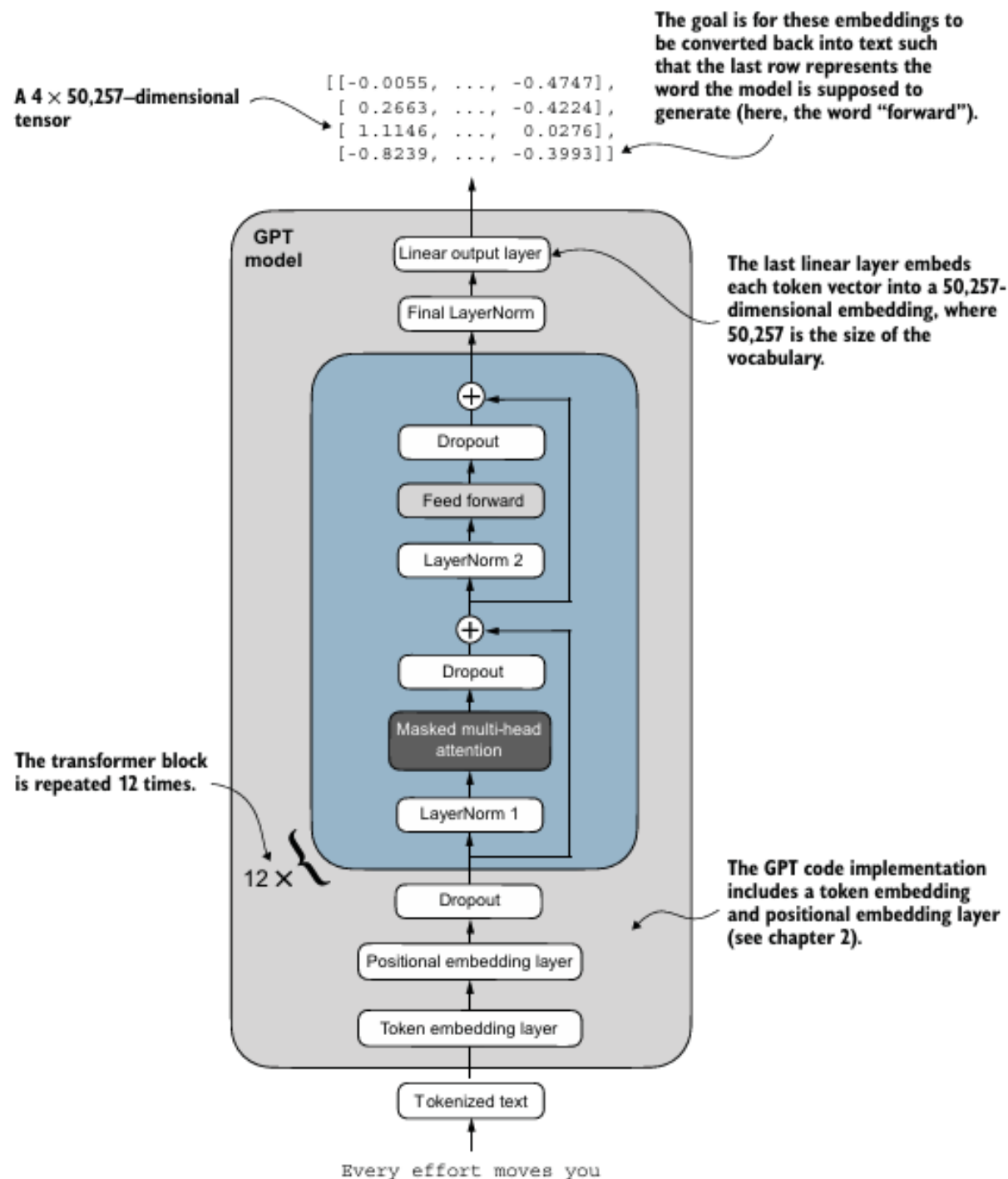
实现 Transformer 模块

- 多头注意力
- 层归一化
- Dropout
- 前馈层和 GELU 激活

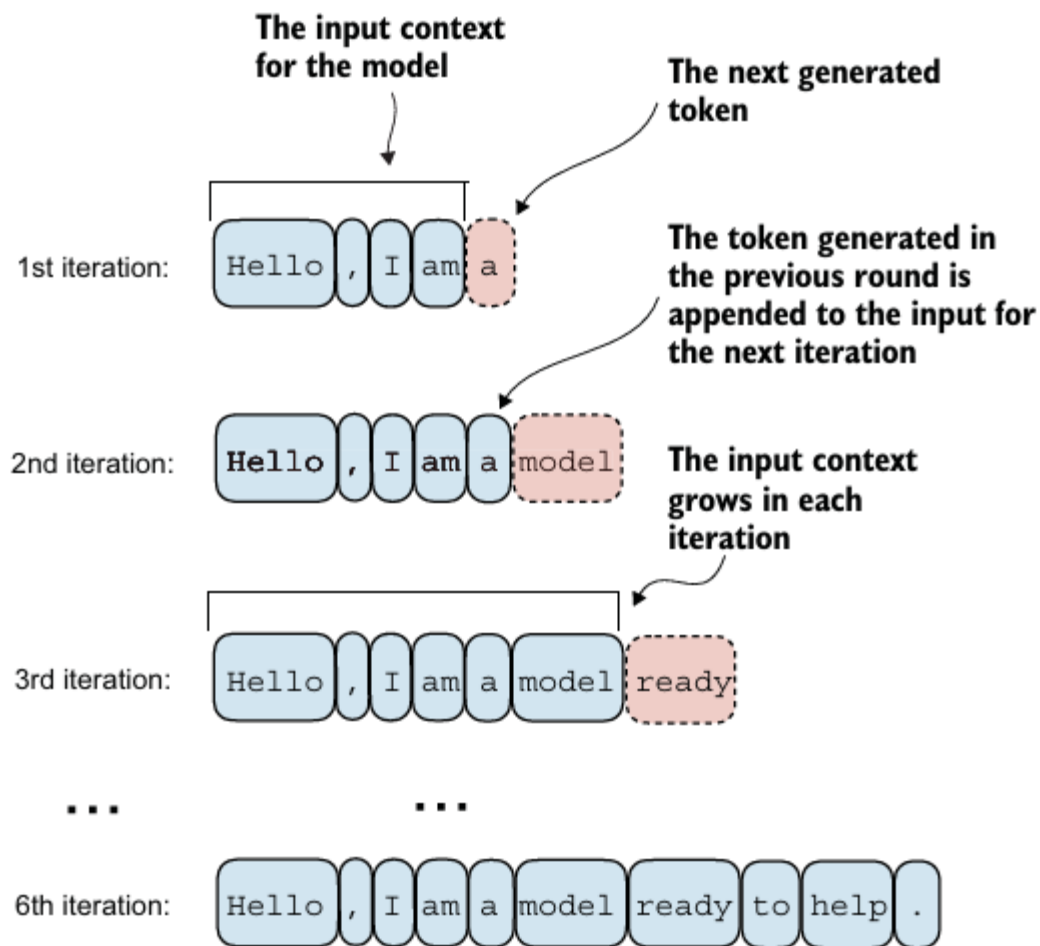


实现GPT-2模型

- 12个Transformer block
- 输出映射到50,257 维， 对应于模型的词汇量）， 以预测序列中的下一个标记。



生成文本：一次一个token地生成文本



1. 将文本输入编码为四个 token ID

2. GPT 模型返回一个由四个向量（行）组成的矩阵，其中每个向量有 50257 个维度（列）。

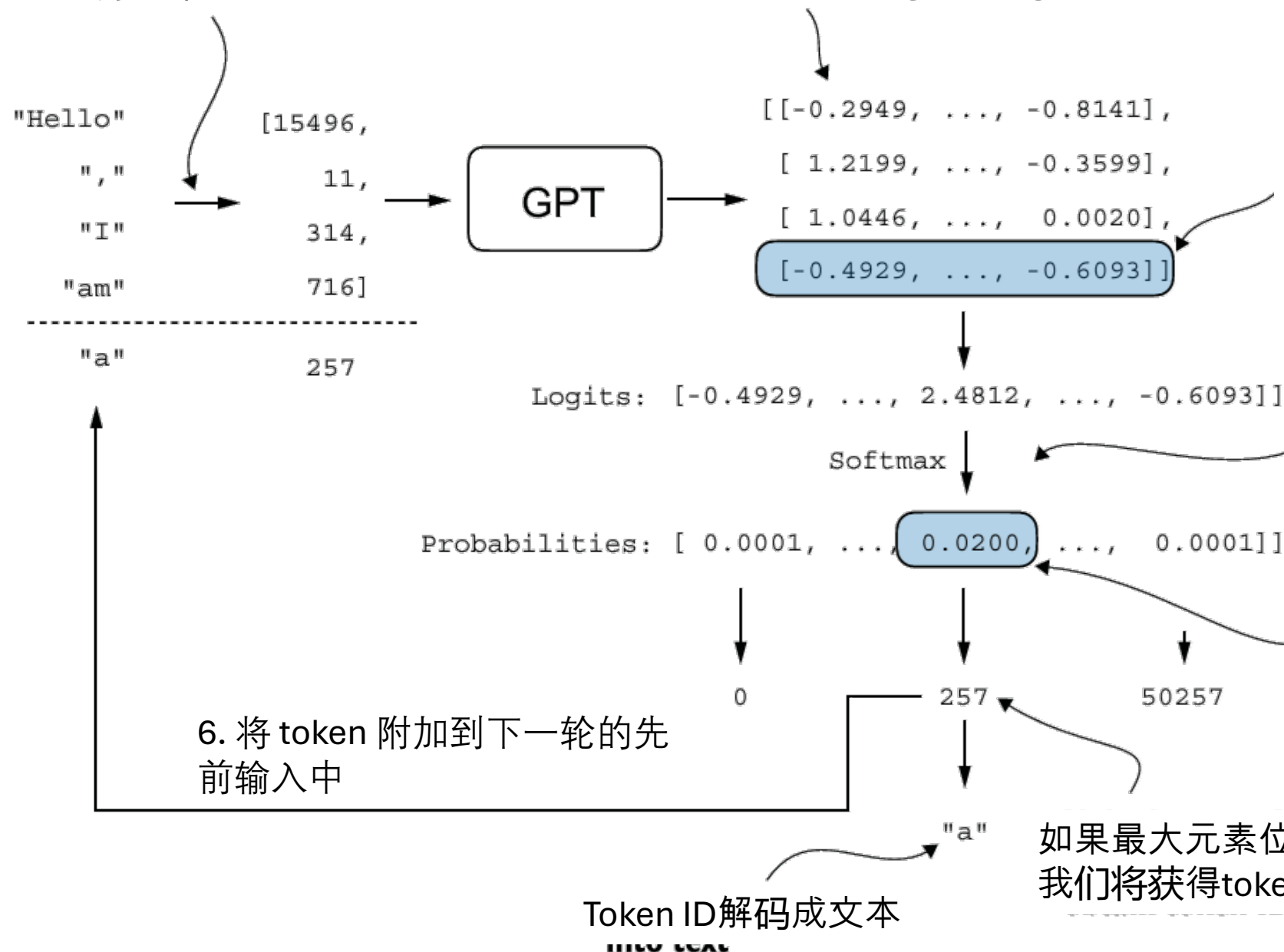
3. 提取最后一个向量，该向量对应于 GPT 模型应该生成的下一个 token

4. 使用 softmax 函数将 logits 转换为概率分布

5. 标识最大值的索引位置，也代表 token ID

6. 将 token 附加到下一轮的先前输入中

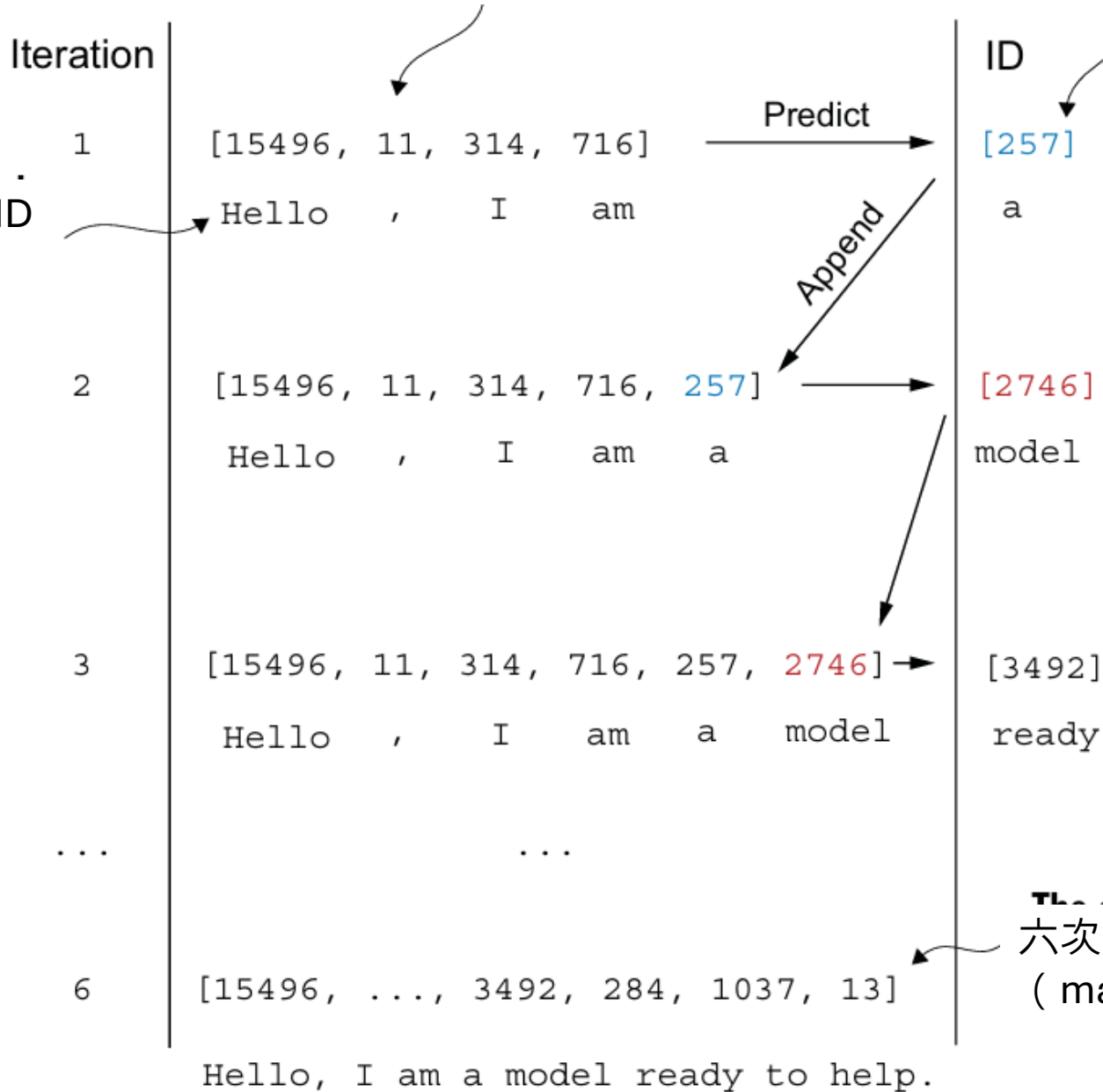
如果最大元素位于位置 257，我们将获得 token ID 257。



作为LLM输入提供的初始
token (上下文)

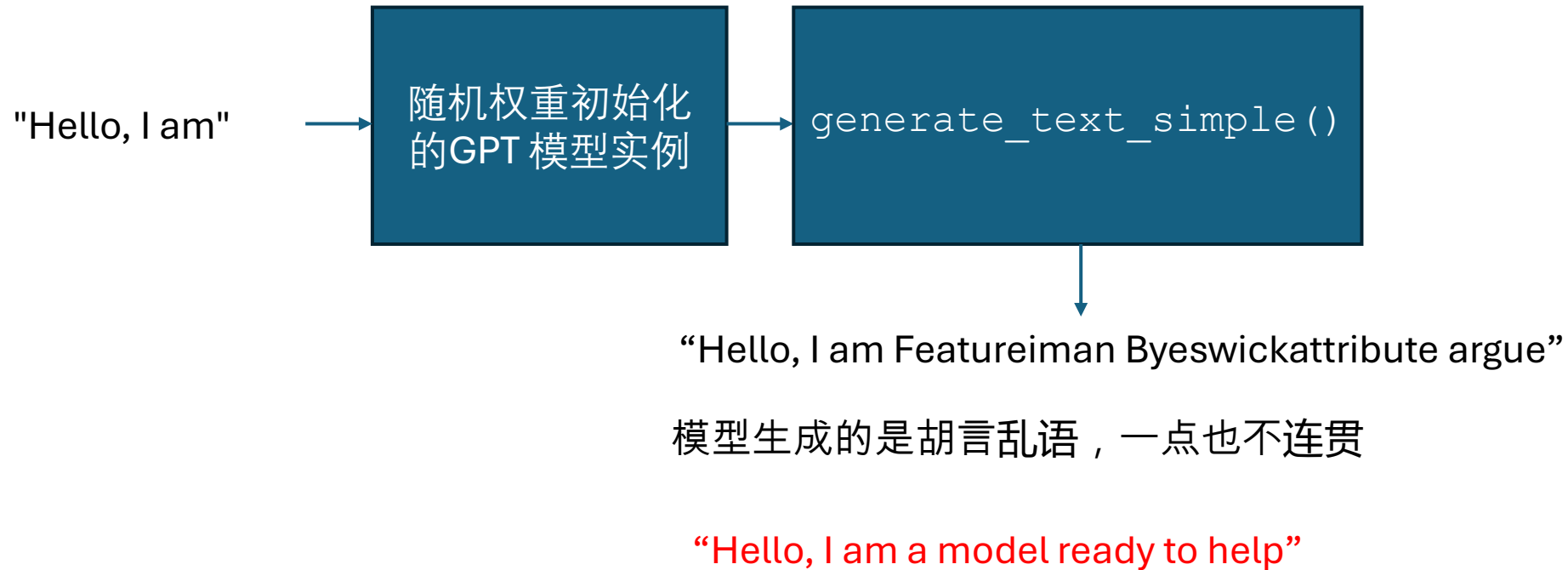
预测的token ID 将附加到
上下文中以供下一轮使
用。

为了便于说明, 将 token ID
转换为文本表示



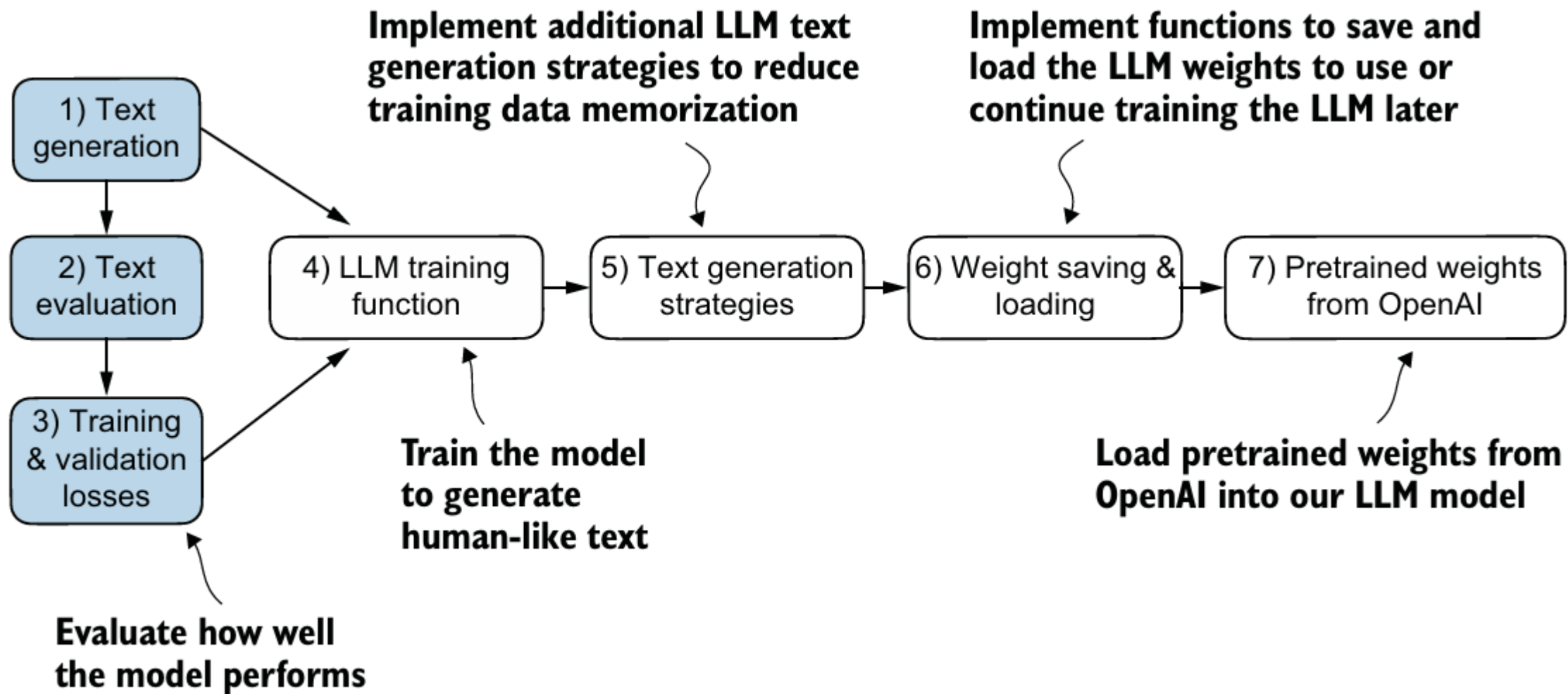
The output tokens
六次迭代后的输出token
(max_new_tokens=6)

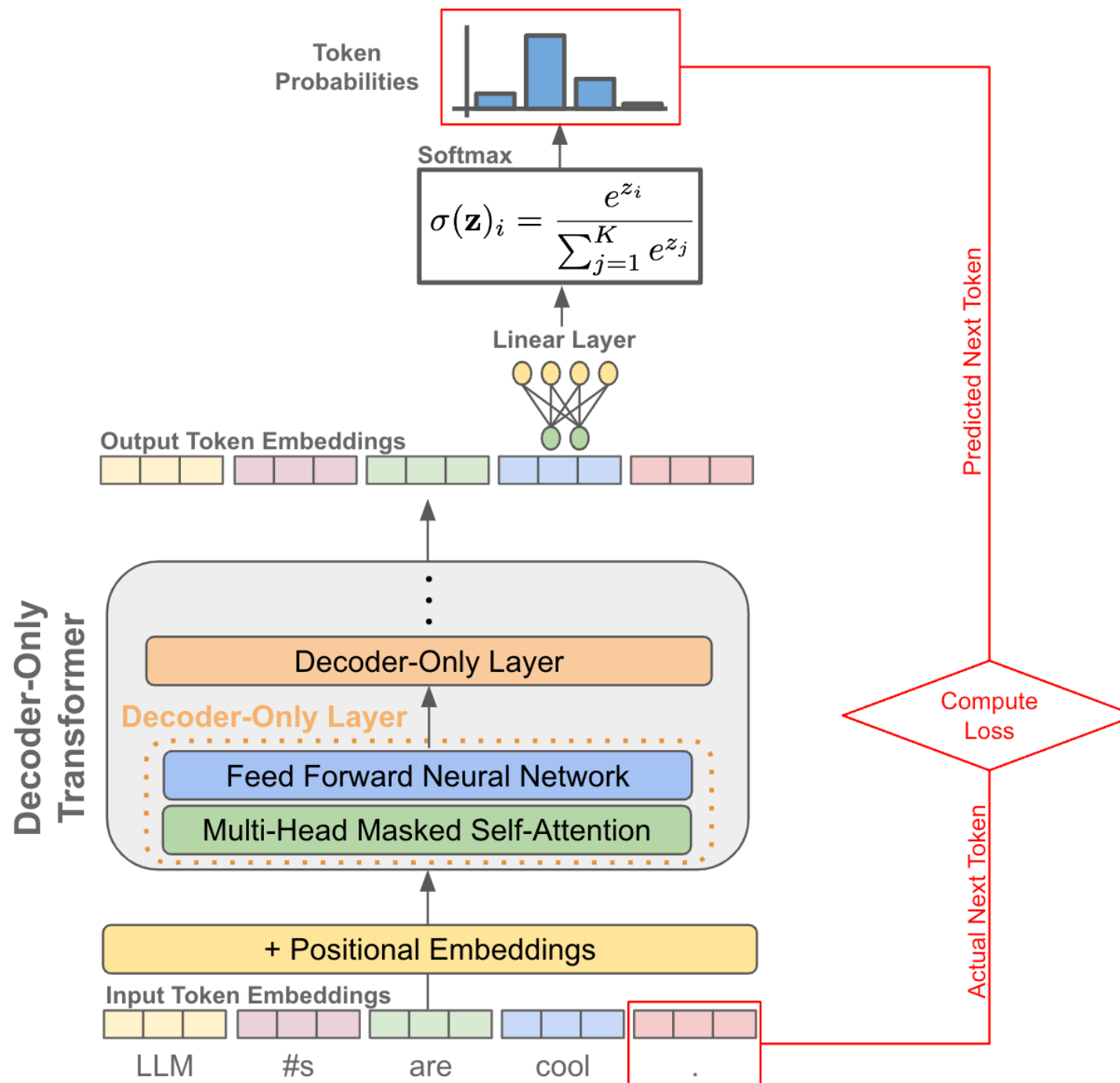
使用随机权重初始化的**GPT**生成文本



在未标记的数据上 预训练GPT 模型

- 计算训练集和验证集损失，以评估训练期间生成的文本的质量
- 实现训练函数并进行预训练
- 保存和加载模型权重以继续训练
- 从 OpenAI 加载预训练权重



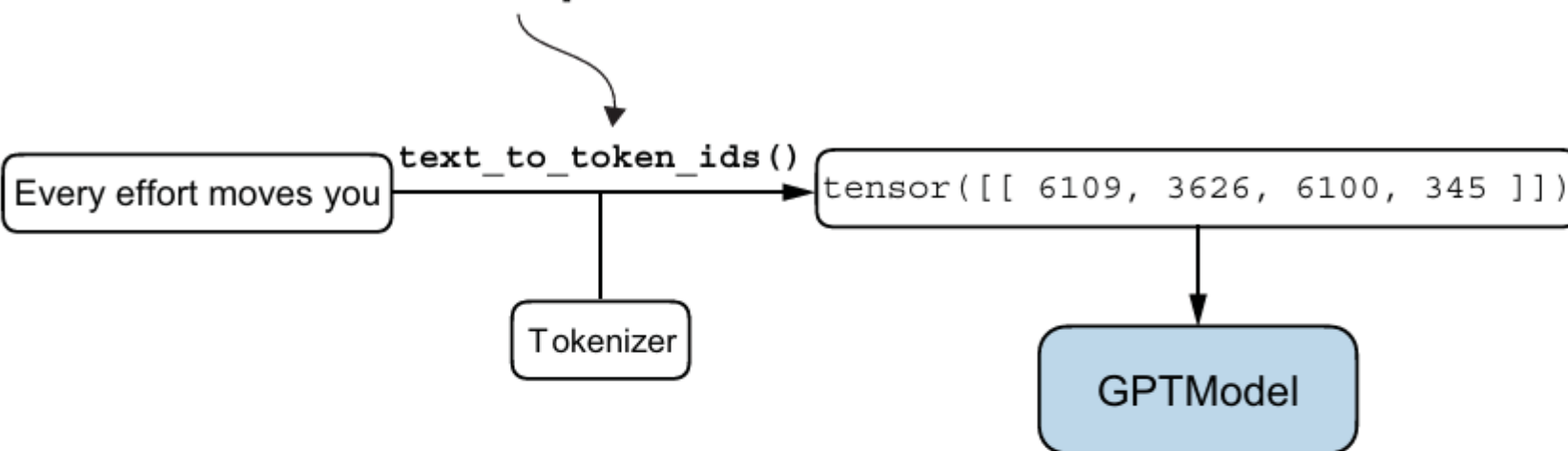


使用 GPT 生成文本



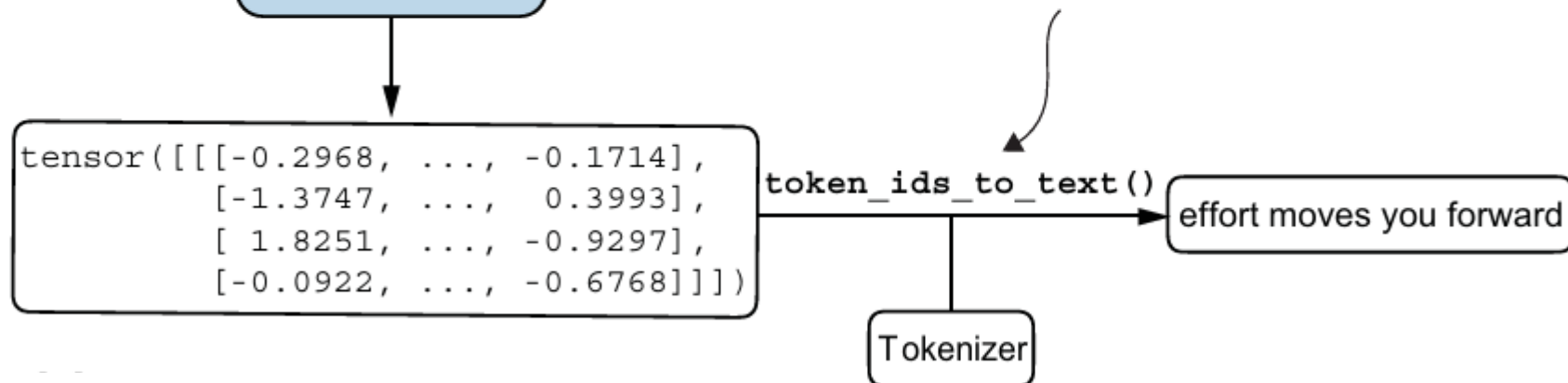
1. 使用分词器将输入文本编码为token ID 表示。

text into a token ID representation.



3. 将 logits 转换为token ID 后，我们使用标记器将这些 ID 解码回文本表示形式。

a text representation.



2. 给定四个输入token ID，该模型将生成 4 个 logit 向量（行），其中每个向量有 50,257 个元素（列），等于词汇表大小。

(columns) equal to the vocabulary size.

从输入文本到 LLM 生成文本



```
vocabulary = {  
  "a": 0,  
  "effort": 1,  
  "every": 2,  
  "forward": 3,  
  "moves": 4,  
  "you": 5,  
  "zoo": 6  
}
```

1. 使用词汇表
映射文本到
token ID

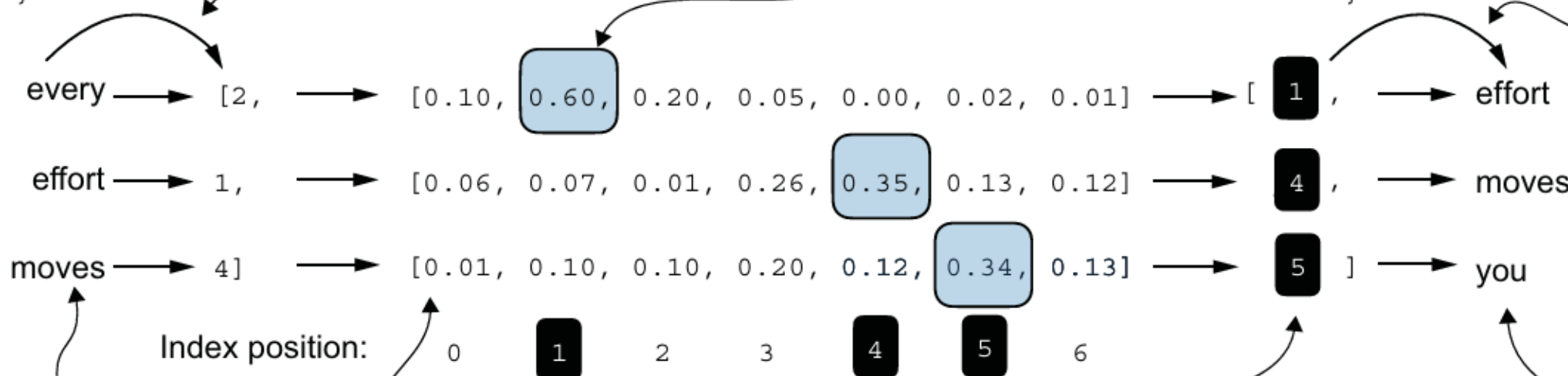
it
is.

3. 在每个行向量中找到概率值最
高的索引位置，这可以通过
argmax完成。
IS DONE VIA THE ARGMAX FUNCTION.

```
inverse_vocabulary = {  
  0: "a",  
  1: "effort",  
  2: "every",  
  3: "forward",  
  4: "moves",  
  5: "you",  
  6: "zoo"  
}
```

5. 通过逆词汇表
将索引转换回文
本。

**the inverse
vocabulary.**



Input text

2. 通过函数获取每个输入
tokens的七维概率行向量。
**for each input token via
the softmax function.**

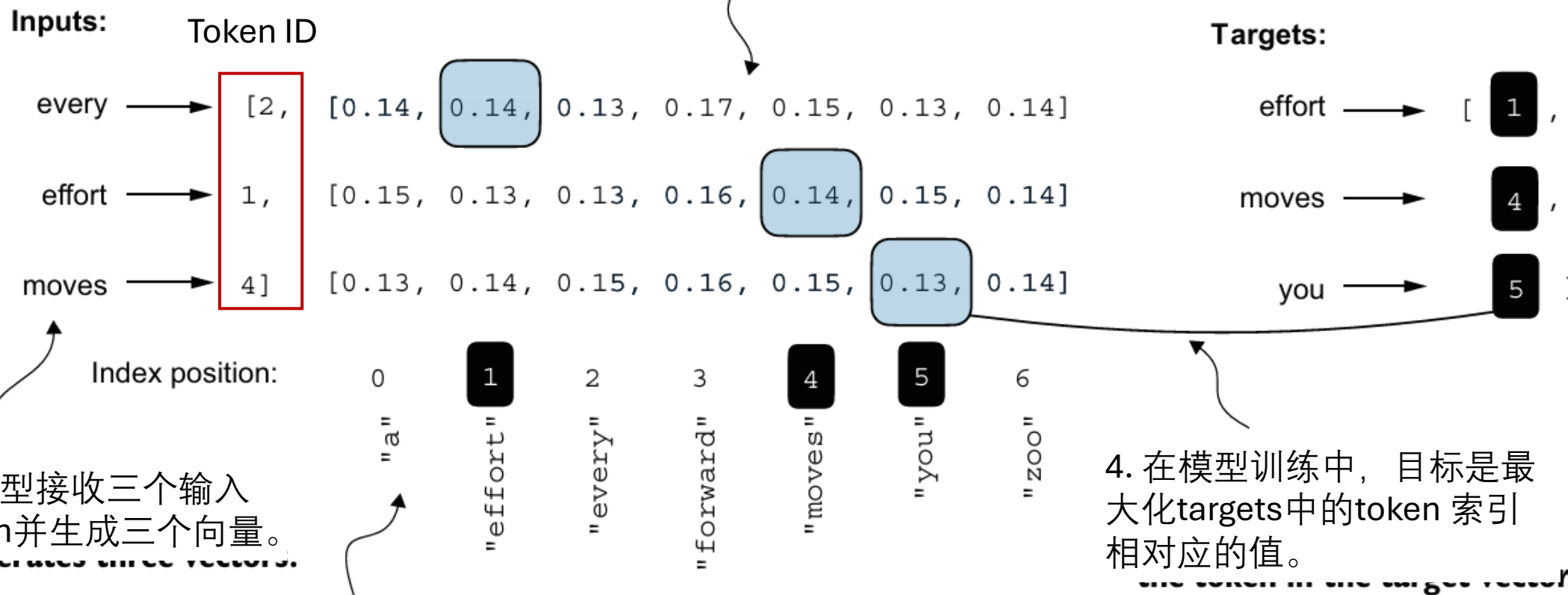
4. 获取所有预测的 token ID 即
概率最高的索引位置。
**positions with the
highest probabilities.**

输出文本
**generated by
the LLM**

使用 GPT 生成文本



3. 未经训练的模型会为
每个标记生成随机向量。
vectors for each token.

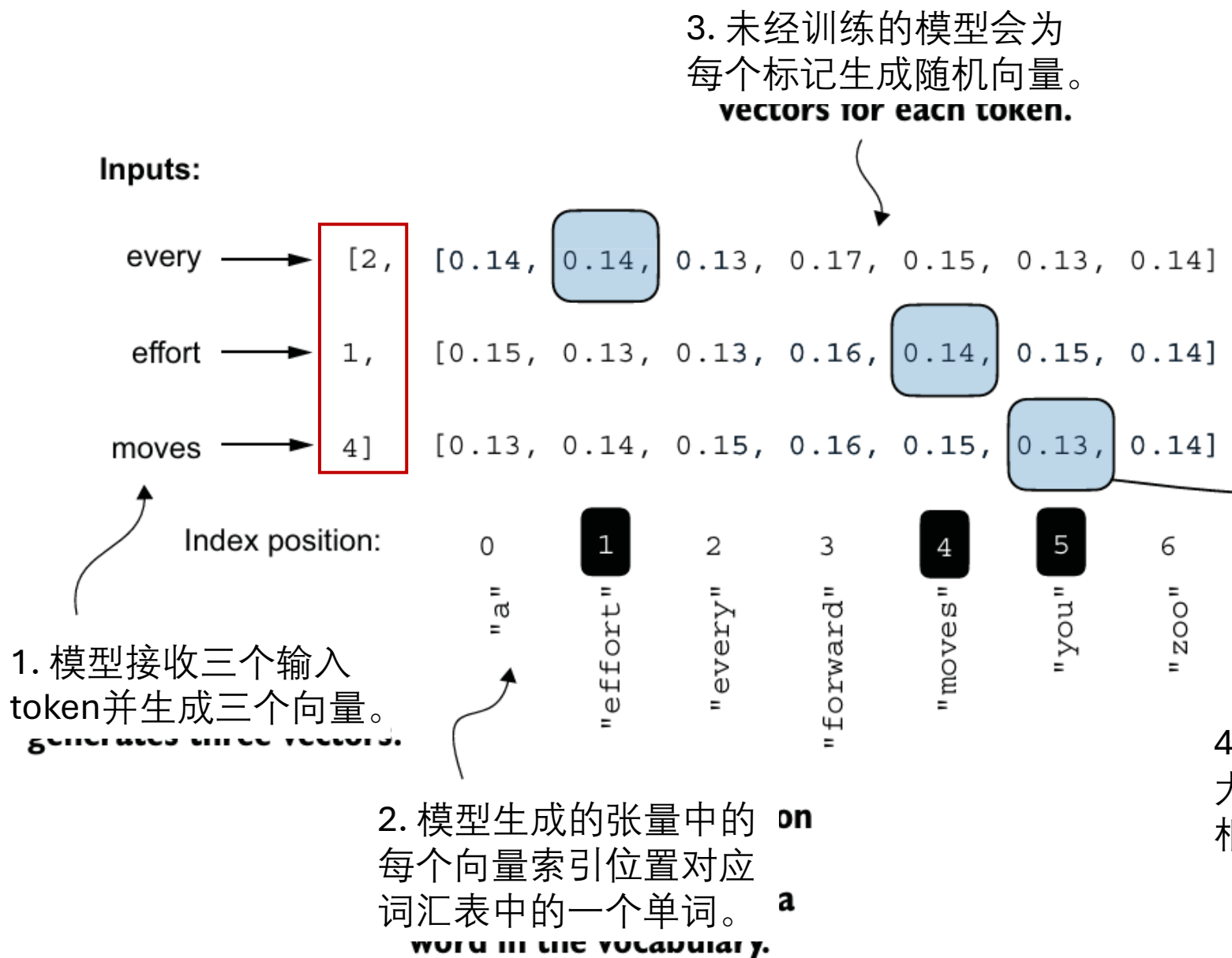


2. 模型生成的张量中的
每个向量索引位置对应
词汇表中的一个单词。
word in the vocabulary.

计算文本生成损失：交叉熵和困惑度

计算交叉熵损失

$$L_i = -\log\left(\frac{e^{sy_i}}{\sum_j e^{s_j}}\right)$$



$$L_1 = -\log(0.13)$$

$$L_2 = -\log(0.13)$$

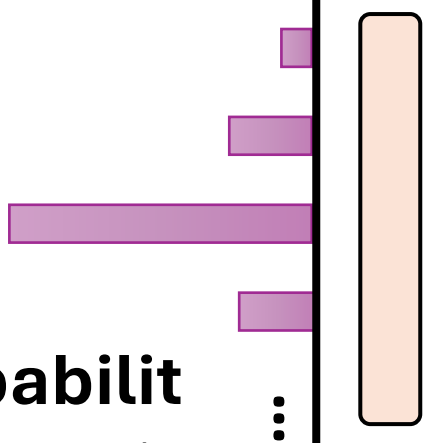
$$L_3 = -\log(0.15)$$

4. 在模型训练中，目标是最大化targets中的token 索引相对应的值。

如何控制 GPT 模型的输出

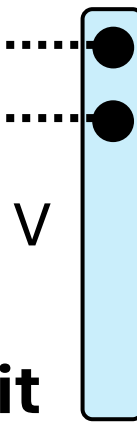
温度缩放

Probability
(0-1, sum to 1)



score of token 0
score of token 1
softmax

Logit
(any number)



GPT Model

概率大

$$\frac{e^{s_1}}{M}$$

$$\frac{e^{s_2}}{M}$$

$$\frac{e^{s_3}}{M}$$

$$e^{s_1}$$

$$e^{s_2}$$

$$e^{s_3}$$

s_1

s_2

s_3

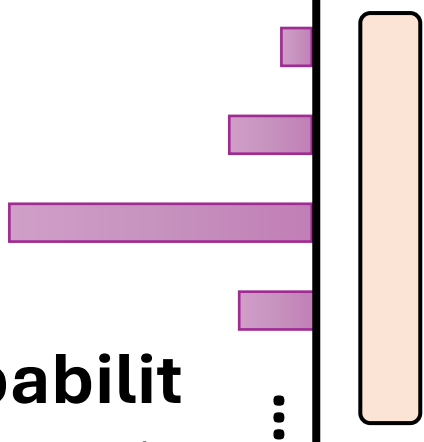
数值大

Logit

$$M = e^{s_1} + e^{s_2} + e^{s_3}$$

温度缩放

Probability
(0-1, sum to 1)



score of token 0
score of token 1
softmax

Logit

(any number)



GPT Model

T 越大
分布越平均
(创意模式)

$$\frac{e^{s_1/T}}{M}$$

$$\frac{e^{s_2/T}}{M}$$

$$\frac{e^{s_3/T}}{M}$$

$$e^{s_1/T}$$

$$e^{s_2/T}$$

$$e^{s_3/T}$$

s_1

s_2

s_3

$$M = e^{s_1/T} + e^{s_2/T} + e^{s_3/T}$$

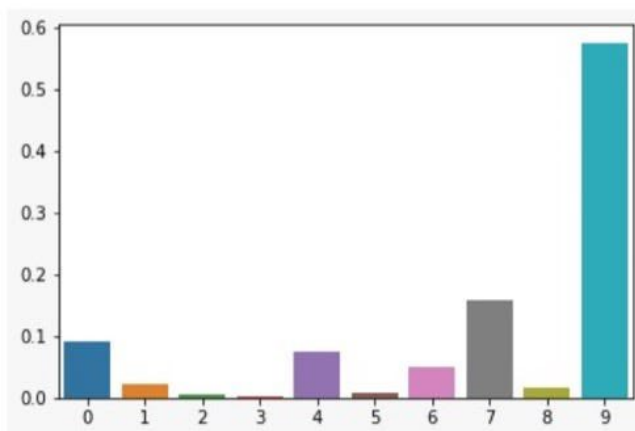
Temperature

Logit

- 语言模型的输出层，我们得到的是每个词的 logits（原始分数）：
 - logits $\rightarrow$ softmax $\rightarrow$ 概率分布 $\rightarrow$ 采样生成 token
 - 温度缩放就是在 softmax 之前，对 logits 除以一个“温度”参数 T:

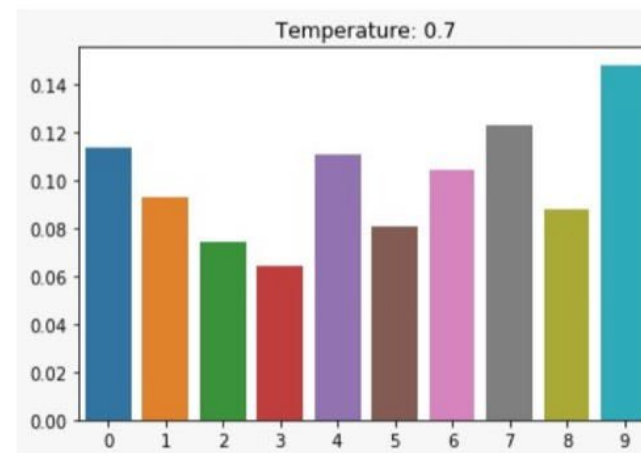
SOFTMAX WITHOUT TEMPERATURE (T=1)

$$\frac{e^{z_i}}{\sum_j e^{z_j}}$$



SOFTMAX WITH TEMPERATURE

$$\frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$



温度缩放-练习



假设 logits = [3.0, 1.0, 0.1] :

使用softmax计算器计算当温度T为0.5, 1.0, 2.0时, softmax的结果

[Softmax Function, Calculator and Formula](#)

温度 T	softmax 结果 (近似)
0.5	
1.0	
2.0	

温度缩放



- 语言模型的输出层，我们得到的是每个词的 logits（原始分数）：
 - logits $\rightarrow$ softmax $\rightarrow$ 概率分布 $\rightarrow$ 采样生成 token
 - 温度控制了 LLM 在生成文本时的“自由度”：**低温更确定，高温更创造。**

温度 T	说明	结果
$T = 1.0$	默认设置，原始 softmax	正常随机性
$T < 1.0$	变“冷”，更确定性	更偏向最大概率的 token（更保守）
$T > 1.0$	变“热”，更随机	更均匀概率、更多样的输出（更creative）

- 语言模型的输出层，我们得到的是每个词的 logits（原始分数）：
 - logits $\rightarrow$ softmax $\rightarrow$ 概率分布 $\rightarrow$ 采样生成 token
 - 温度控制了 LLM 在生成文本时的“自由度”：**低温更确定，高温更创造。**

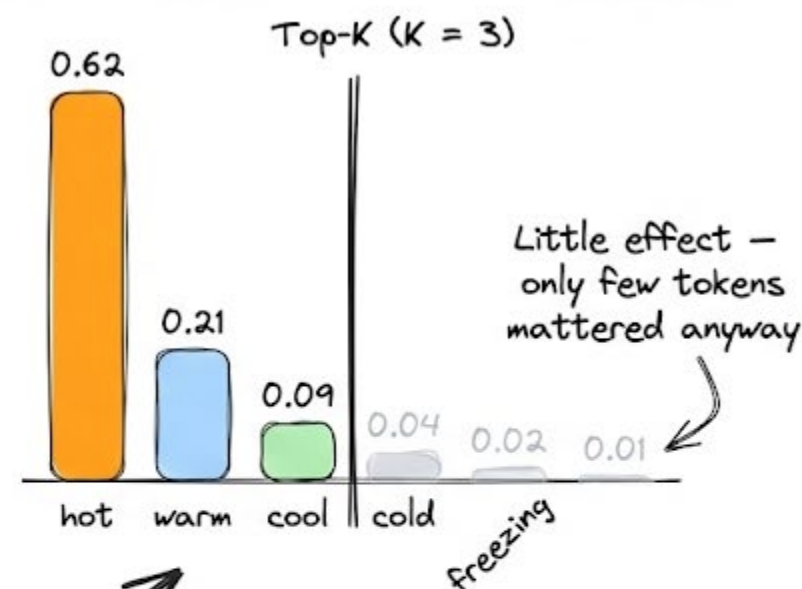
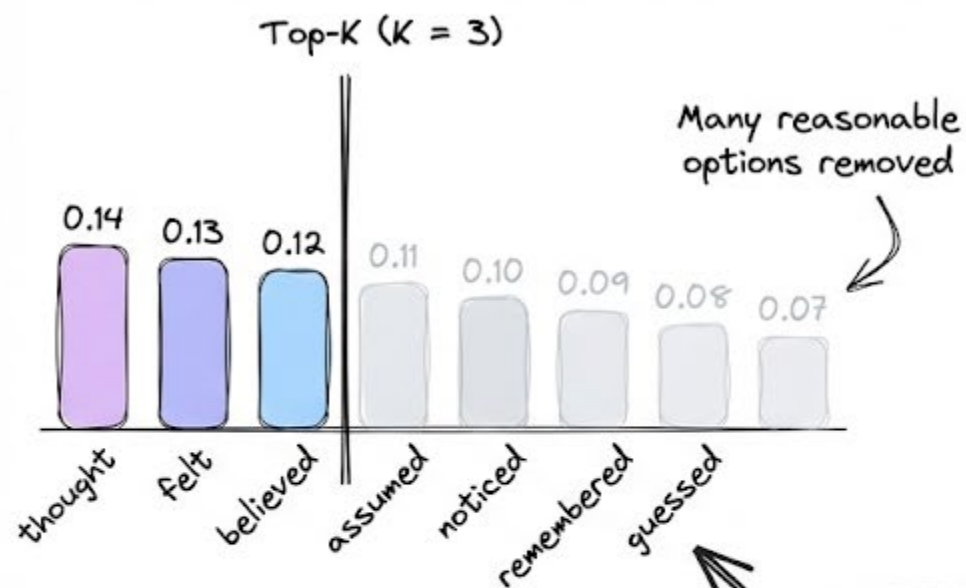
阶段	是否使用温度	为什么使用
训练阶段	×	因为训练目标是让模型学会预测 ground-truth，目标稳定
生成阶段	√	控制输出多样性，探索 vs 保守，creative vs safe

Top-k 采样的基本思路：只在概率最高的前 K 个 token 中随机采样。其他的全部置为 0。

Vocabulary:	"closer"	"every"	"effort"	"forward"	"inches"	"moves"	"pizza"	"toward"	"you"
Index position:	0	1	2	3	4	5	6	7	8
Logits	= [4.51,	0.89,	-1.90,	6.75,	1.63,	-1.62,	-1.89,	6.28,	1.79]
↓									
Top-k (k = 3)	= [4.51,	0.89,	-1.90,	6.75,	1.63,	-1.62,	-1.89,	6.28,	1.79]
↓									
-inf mask	= [4.51,	-inf,	-inf,	6.75,	-inf,	-inf,	-inf,	6.28,	-inf]
↓									
Softmax	= [0.06,	0.00,	0.00,	0.57,	0.00,	0.00,	0.00,	0.36,	0.00]

Top-K Sampling (基于数量的采样)

- 核心理想 (左侧)
 - 从概率最高的前 K 个 token 中采样
 - 其余 token 概率置为 0
 - 再重新归一化进行采样



Same K behaves very differently depending on the distribution shape

Broad / Flat Distribution

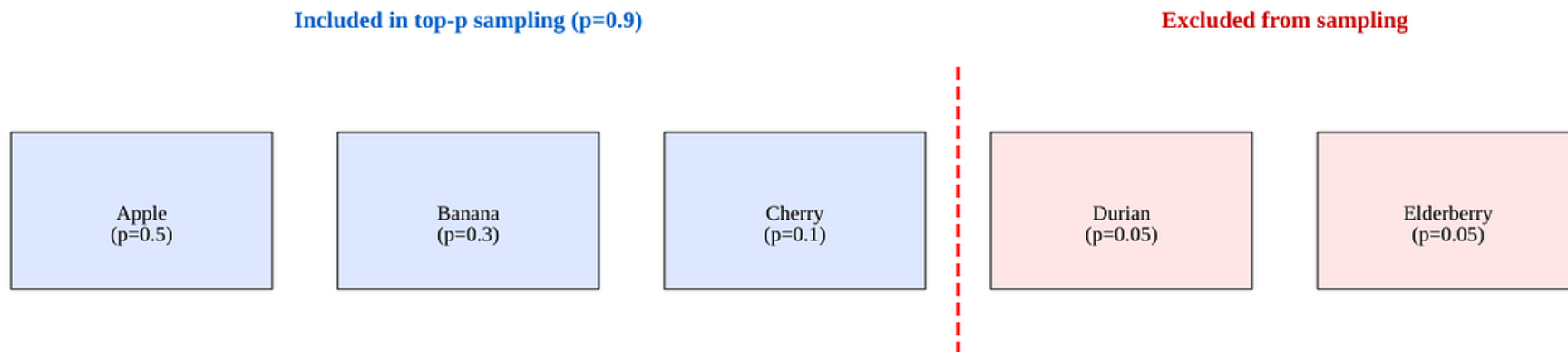
Narrow / Peaked Distribution

Top-P Sampling

(Nucleus Sampling, 基于概率的采样)

- 核心思想
 - 按概率从大到小排序
 - 累计概率直到 $\geq p$
 - 选取最小集合进行采样

Top-P Sampling with $p=0.9$



Cumulative probability: $0.5 + 0.3 + 0.1 = 0.9 \geq p$

练习

- 假设模型要预测下一个词，给定某语言模型输出的 logits:
- 已知:
 - Temperature:
 - T=2 (高温)
 - T=1 (标准)
 - T=0.5 (低温)
 - Top-K: k = 2
 - Top-P: p = 0.8
 - Softmax 定义:

$$P_i = \frac{e^{z_i}}{\sum e^{z_j}}$$

Token	Logit
A	2
B	1
C	0
D	-1

练习

Softmax Function, Calculator and Formula

• 问题

(1) 分别写出三个温度下的 **softmax 概率** (先缩放 logits)

Token

T=2

T=1

T=0.5

A

B

C

D

练习

Softmax Function, Calculator and Formula

• 问题

(1) 分别写出三个温度下的 **softmax 概率** (先缩放 logits)

Token	T=2	T=1	T=0.5
A	1	2	4
B	0.5	1	2
C	0	0	0
D	-0.5	-1	-2

练习

- 问题

(2) 在 **Temperature = 0.5, 1, 2** 的情况下, Top-K ($k=2$) 的结果、

(3) 在 **Temperature = 0.5, 1, 2** 的情况下:

应用 Top-P ($p=0.8$) , 写出保留的 token